



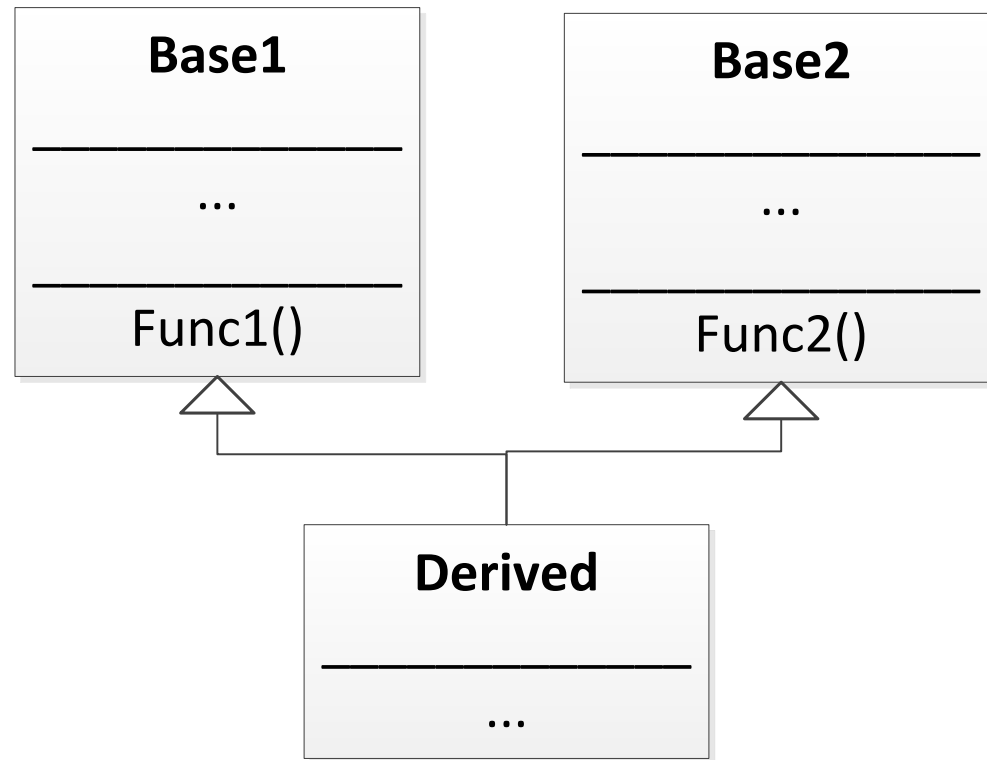
CS1004 – Object Oriented Programming

Lecture # 14
Monday, April 04, 2022
Spring 2022
FAST – NUCES, Faisalabad Campus

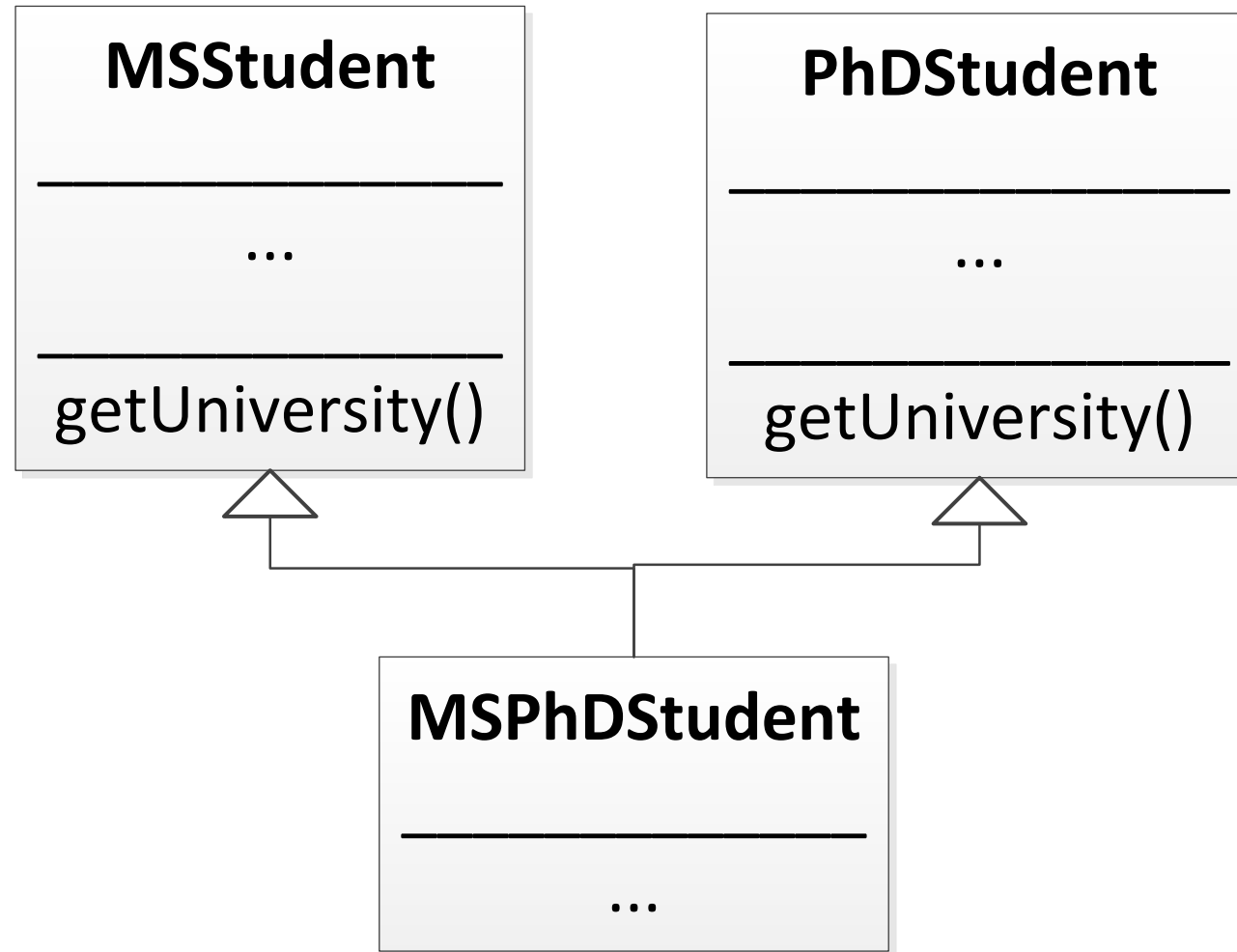
Rizwan Ul Haq

Multiple Inheritance

- A class can inherit from more than one class



Multiple Inheritance



Multilevel Inheritance

```
class MPhDStudent:  
    public MSStudent,  
    public PhDStudent  
{  
  
};
```

Multiple Inheritance

- ➡ Derived class can inherit from public base class as well as private and protected base classes

```
class MPhDStudent:  
    private MStudent,  
    private PhDStudent  
{  
  
};
```

Multiple inheritance

- The derived class inherits data members and functions from all the base classes
- Object of derived class can perform all the tasks that an object of base class can perform

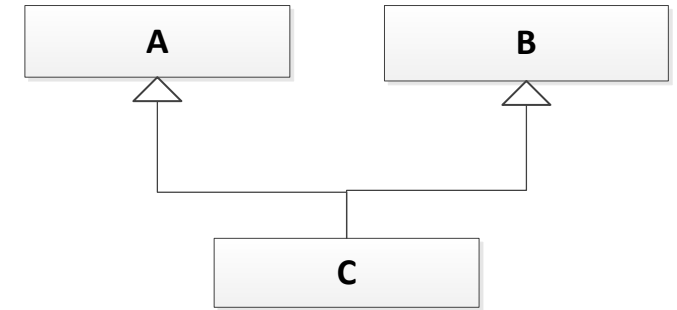
Multiple inheritance: Example

```
int main()
{
    MSPhDStudent obj1;
    obj1.getMSThesis();
    obj1.getPhDThesis();
}
```

Multiple inheritance

```
class A {  
public:  
    A() {  
        cout << "Constructor of the A  
                called";  
    }  
};  
class B {  
public:  
    B() {  
        cout << "Constructor of the A  
                called";  
    }  
};
```

```
int main()  
{  
    C c;  
    return 0;  
}
```



Output:

Constructor of the A called
Constructor of the B called

Multiple inheritance: Ambiguity

- If more than one base class have a function with same signature then the child will have two copies of that function
- Calling such function will result in ambiguity

Multiple inheritance: Ambiguity

```
class MSStudent{
    int degLength;
public:
    int getDegLength()
    {
        return degLength;
    }
};
class PhDStudent {
    int degLength;
public:
    int getDegLength()
    {
        return degLength;
    }
};
```

```
class MPhDStudent:public MSStudent,
public PhDStudent {

};
int main() {
    MPhDStudent obj1;
    obj1.getDegLength(); //Ambiguous
                        //access
}
```



"MPhDStudent::getDegLength" is ambiguous



[C2385](#)

ambiguous access of 'getDegLength'

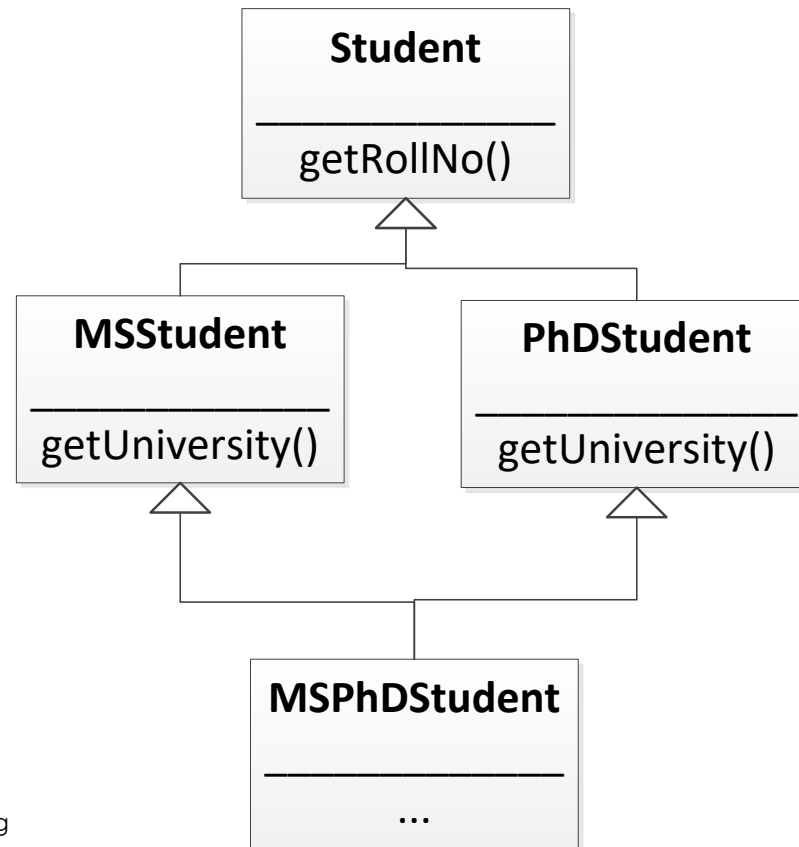
Multiple inheritance: Ambiguity

- ➡ The developer must explicitly specify the class name when calling a function

```
int main() {  
    MSPhDStudent obj1;  
    cout << obj1.MSStudent::getDegLength(); //Access MSStudent's function  
    cout << obj1.PhDStudent::getDegLength(); //Access PhDStudent's function  
}
```

Hybrid inheritance

- The ambiguous call problem can arise when dealing with multiple level of multiple inheritance



Hybrid inheritance: Example

```
class Student {
    int rollNo;
public:
    int getRollNo() {
        return rollNo;
    }
};
class MSStudent:public Student{
};
class PhDStudent :public Student {
};
```

```
class MPhDStudent:
    public MSStudent,
    public PhDStudent {
};
int main() {
    MPhDStudent obj1;
    obj1.getRollNo(); //Ambiguous
                      //access
}
```



"MPhDStudent::getRollNo" is ambiguous



C2385

ambiguous access of 'getRollNo'

Hybrid inheritance

- ➡ Data member must be used with care when dealing with more than one level of inheritance

Hybrid inheritance

```
class Student {  
protected:  
    int age;  
public:  
    int getAge() {  
        return age;  
    }  
};
```

```
-----  
class MSStudent:public Student{  
};
```

```
-----  
class PhDStudent :public Student {  
};
```

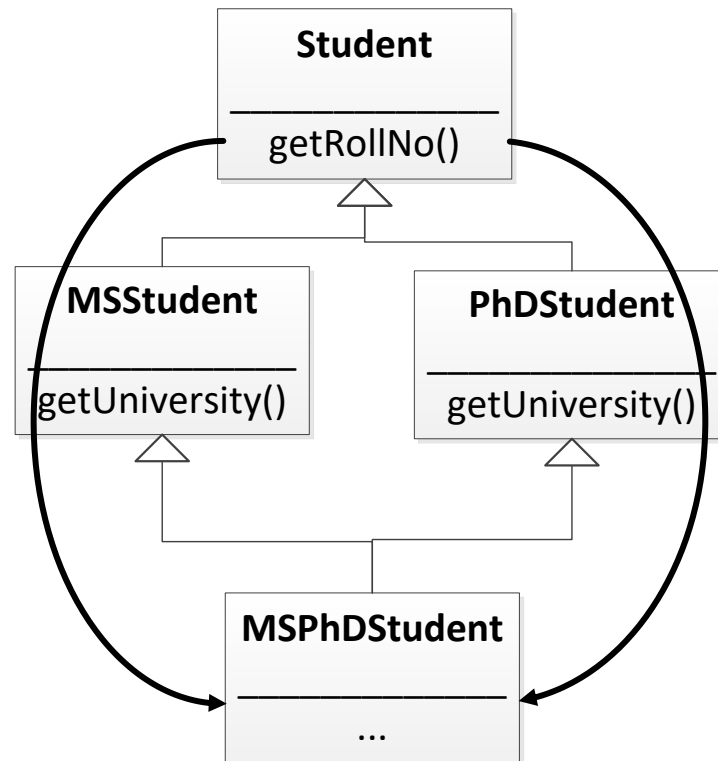
```
class MPhDStudent:public MSStudent, public  
PhDStudent {  
public:  
    MPhDStudent() {  
        MSStudent::age = 30;  
        PhDStudent::age = 35;  
    }  
};
```

```
-----  
int main() {  
    MPhDStudent obj1;  
    cout << obj1.MSStudent::getAge();  
    cout << obj1.PHDStudent::getAge();  
}
```

There are multiple copies of data member age

Diamond Problem

- When from an indirect inheritance a function is inherited from more than one paths
- The compile cannot determine which path to choose



Memory View

Data members of Student	Data members of Student
Data members of MSStudent	Data members of PhDStudent
Data members of MSPhDStudent	

Virtual Inheritance

- ➡ In virtual inheritance there is exactly one copy of the anonymous base class object

Virtual inheritance: Example

```
class MSStudent:public virtual Student{  
};
```

```
-----  
class PhDStudent :public virtual Student {  
};
```

```
-----  
class MPhDStudent:public MSStudent, public PhDStudent {  
public:  
    MPhDStudent() {  
        age = 30;  
    }  
};
```

Memory View

Data members of Student	
Data members of MSStudent	Data members of PhDStudent
Data members of MSPhDStudent	

Questions

